



Deep Learning: Rapport de projet

Comparaison des modèles de Computer Vision : CNN
et Transformers

Sixto Cerna
18 mars 2026

Contents

1	Introduction	2
2	Modèles étudiés	2
2.1	Convolutional Neural Networks	2
2.2	Vision Transformers	6
3	Comparaison de performances	7
3.1	Sélection des datasets	8
3.1.1	Dataset A: CNN perdant	8
3.1.2	Dataset B: ViT perdant	8
3.2	Entraînement des modèles	9
3.3	Résultats	10
3.4	Limitations	11
4	Conclusion	12
5	Code	12

1 Introduction

Dans ce rapport, nous allons nous intéresser à comprendre les différences entre les deux architectures les plus importantes dans le domaine du *Computer Vision*. Ce domaine regroupe plusieurs tâches : segmentation d'images, reconnaissance d'objets, entre autres; mais nous allons nous concentrer sur la tâche de classification d'images. Celle-ci consiste à attribuer un label à une image donnée (par exemple associer le label "chien" à une image quand un chien est effectivement présent sur l'image). L'objectif principal est comprendre comment les différences entre les Vision Transformers (ViT) et les Convolutional Neural Networks (CNN) peuvent rendre l'une de ces architectures plus adaptée à un jeu de données. Dans une première partie, nous allons présenter le fonctionnement de chacune de ces architectures. Dans une deuxième partie, nous allons mettre en évidence un dataset où les ViT sont plus performants que les CNN et un dataset où les CNN sont plus performants que les ViT.

2 Modèles étudiés

Dans cette section nous allons présenter les CNN et les ViT. Nous nous plaçons dans le contexte de classification. Autrement dit, l'entrée du réseau est une image de taille $\mathbb{R}^{W \times H \times C}$ et la sortie du réseau est un vecteur de $\mathbb{R}^L$ où :

- W (Width) est la largeur de l'image
- H (Height) est la hauteur de l'image
- C (Channels) est le nombre de canaux de l'image. Par exemple une image en blanc et noir possède un seul canal (l'intensité du pixel) et une image en RGB possède 3 canaux: un pour l'intensité du rouge, un pour l'intensité du bleu et un pour l'intensité du vert.
- L est nombre de classes

Un MLP (Multi Layer Perceptron) classique aurait besoin de $W \times H \times C$ paramètres par neurone à sa première couche. Pour une image en couleurs de 220x220 pixels, le réseau possède 145K paramètres par neurone à la première couche. De plus, cette approche ignore la structure intrinsèque aux images. Dans un problème tabulaire permuter l'ordre des colonnes ne devrait pas changer la sortie; en revanche, permuter l'ordre des pixels dans une image change complètement le label. Ainsi, les MLP typiques ne sont pas adaptés pour traiter les images parce que d'une part les MLPs ignorent la structure intrinsèque aux images et d'autre part le nombre de paramètres augmente très vite avec la taille de l'image.

2.1 Convolutional Neural Networks

Les CNN (Convolutional Neural Networks) ont été introduits pour remédier les problèmes induits par l'approche naïve des MLP dans le contexte du traitement

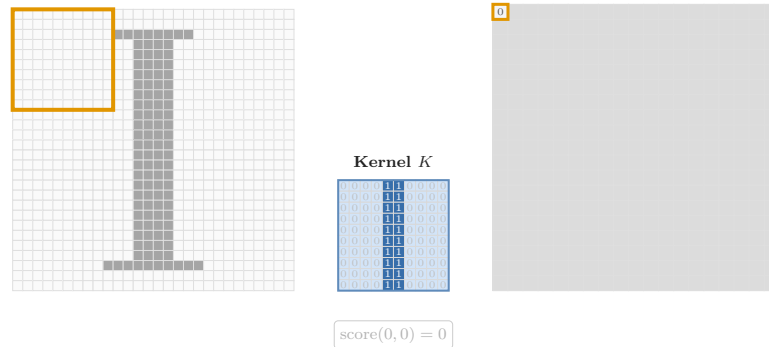


Figure 1: À gauche : image de départ avec le chiffre "1". Le carré orange représente la position du kernel à la première étape de la convolution. Au centre: le kernel, le motif recherché sous forme de matrice. À droite: le résultat partiel de la convolution. La case orange correspond à la valeur obtenue en appliquant le kernel sur la zone orange de l'image d'origine.

d'images.

Pour motiver l'idée clé derrière les CNN, considérons l'exemple suivant. Nous essayons de distinguer des "1" et des "0". Étant donné la différence très marquée entre les deux symboles qu'on cherche à reconnaître, nous pouvons considérer que chercher des motifs simples peut suffire. Par exemple, si l'on détecte une ligne verticale il est très probable qu'il s'agisse du chiffre "1". Cette approche pour l'instant ne repose pas sur la recherche de paramètres optimaux. En revanche, deux questions se posent. D'une part, où faut-il chercher la ligne verticale? Nous ne connaissons pas en amont l'emplacement précis de notre objet. D'autre part, il se peut que la ligne ne soit pas parfaitement verticale. En effet, si les chiffres sont écrits à la main il est très probable que les lignes ne soient pas parfaitement verticales.

Pour résoudre ces deux difficultés nous allons au lieu de chercher exactement une ligne verticale, nous allons chercher des motifs qui *ressemblent* à des lignes verticales. Pour cela, nous allons représenter le motif cherché comme une matrice de taille $k \times k$ (appelé kernel) et nous allons le glisser sur l'image en comparant à chaque position le motif contre la zone observée de l'image à cette position. Une manière de quantifier la similarité entre le contenu de l'image à cette position et le motif est de sommer les composantes du produit de Hamming entre ces deux éléments. Dans le cas des images binaires (i.e. en noir et blanc) ceci correspond au nombre de pixels qui sont colorés aux mêmes positions. Cette opération est appelée convolution entre l'image et le kernel, le résultat est une matrice de dimension plus petite que l'image de départ. La procédure est illustrée dans les figures 1, 2, 3 et 4.

Dans ces figures nous observons que faire une convolution permet d'identifier

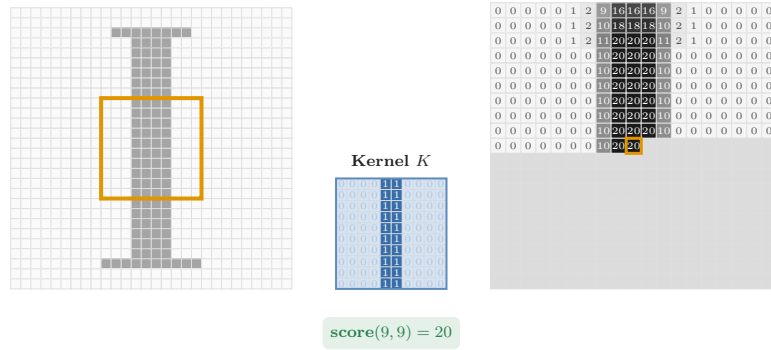


Figure 2: À gauche : l'image de départ avec le chiffre "1". Le carré orange représente la position du kernel. Au centre: le kernel, le motif recherché sous forme de matrice. À droite: le résultat partiel de la convolution de l'image par le kernel. La case orange correspond à la valeur obtenue en appliquant le kernel sur la zone orange de l'image d'origine.

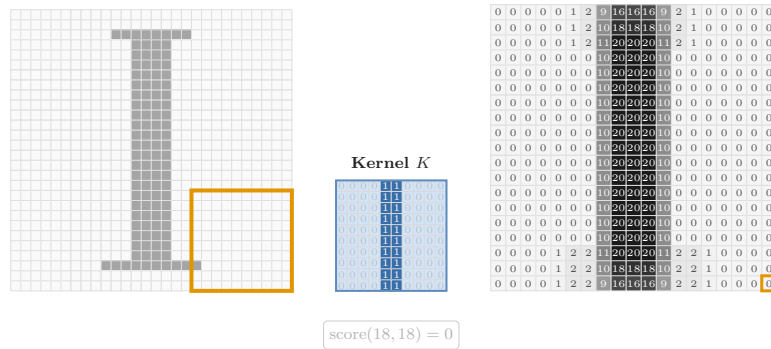


Figure 3: À gauche : l'image de départ avec le chiffre "1". Le carré orange représente la dernière position du kernel. Au centre: le kernel, le motif recherché sous forme de matrice. À droite: le résultat final de la convolution de l'image par le kernel.

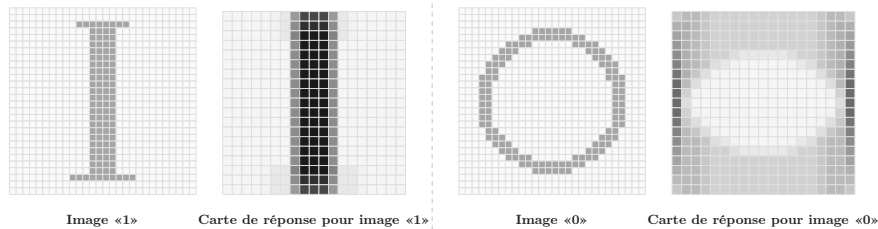


Figure 4: À gauche: l'image de départ "1" avec le résultat de la convolution. À droite: l'image de départ "0" avec le résultat de la convolution. Les cases les plus foncées correspondent à une forte correspondance entre le motif cherché et la zone observée dans l'image de départ à cette position.

les zones où se trouve un motif similaire à celui du kernel. Ici, il est facile de trouver un kernel (ou motif) qui permettra de différencier les images facilement, néanmoins ce n'est pas toujours le cas (e.g. lorsqu'on essaye d'identifier des chats ou des chiens). Par conséquent, les entrées de ces kernels deviennent des paramètres du réseau et on laisse le réseau déterminer les motifs qui permettent de mieux différencier les images.

L'idée clé derrière les CNN est d'introduire des nouvelles types de couches dites de "convolution" qui vont appliquer plusieurs kernels sur les images et qui vont permettre de trouver des "motifs" qui permettent de différencier les images, à l'instar de l'exemple présenté précédemment. Ces couches sont bien adaptées aux images pour plusieurs raisons. D'une part, elles préservent la structure originelle de l'image, i.e. les pixels qui sont proches sont considérés comme appartenant au même ensemble. De plus, cette approche permet de rester invariant par translation: sur le "feature map" (le résultat de la convolution) on observera des activations indépendamment de la position où se trouve le motif. D'autre part, nous avons réduit de manière significative le nombre de paramètres nécessaires. En effet, les paramètres sont uniquement les entrées du kernel. Finalement, l'application successive de ce type de couche permet de réduire la dimension de l'entrée et d'agréger des motifs. Par exemple, les premières couches convolutives vont détecter les lignes horizontales, diagonales, etc. et les couches convolutives suivantes vont les agréger ces signaux pour détecter des formes plus complexes. Ces signaux plus complexes, mais de dimension plus petite que l'image de départ, peuvent être passés à un MLP traditionnel.

Dans la pratique, les couches convolutives sont intercalées avec des couches de pooling qui prennent le maximum ou le minimum dans une zone du feature map. Des connexions résiduelles sont souvent ajoutées aussi pour améliorer la stabilité lors de l'entraînement. De plus, lorsqu'on a une image RGB le kernel regarde les trois canaux en même temps. Une schéma d'un réseau convolutif est présenté dans la figure 5.

Cette architecture fait une hypothèse très importante : la localité. C'est-à-

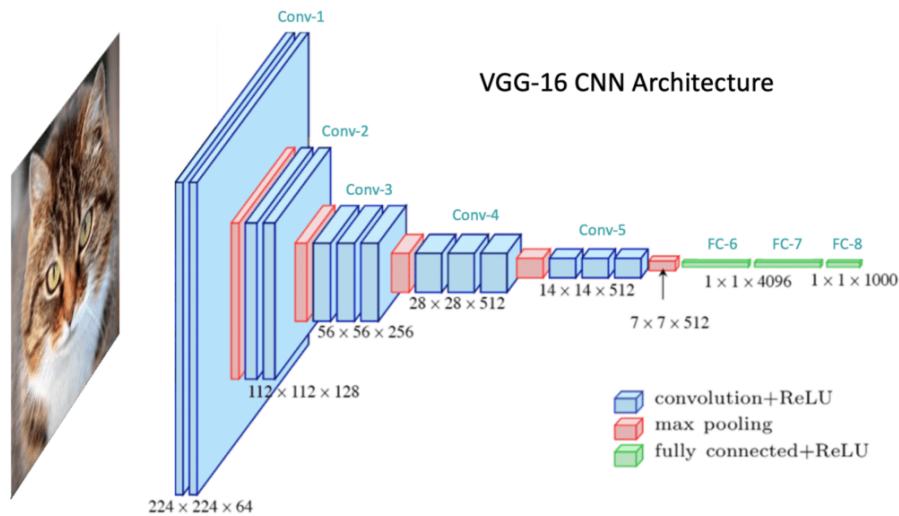


Figure 5: Schema d'un CNN (<https://learnopencv.com/wp-content/uploads/2023/01/Convolutional-Neural-Networks-1024x611.png>)

dire que regarder une zone limitée de l'image suffit à extraire des informations utiles à la classification. En particulier, les CNN vont détecter plus facilement des textures et non des silhouettes (qui demandent d'observer une zone plus large).

2.2 Vision Transformers

Bien que l'hypothèse de localité soit cohérente dans le cadre du traitement d'images, elle n'est pas toujours souhaitable. En effet, il peut avoir des cas où il est cruciale d'être capable d'établir des dépendances entre des régions éloignées de l'image. Par exemple, avec un kernel de taille réduite il peut être difficile de distinguer une voiture d'un camion car la "texture" perçue sera toujours très similaire. Dans ce cas, il faut être capable de mettre en commun des informations issues de positions différentes.

Les ViT (Vision Transformers) utilisent l'architecture des transformers, largement appliqué dans le domaine du NLP, avec des modifications minimales pour traiter des images. En entrée un ViT reçoit de patch d'images, i.e. une image découpée en cases. Chaque case joue le rôle d'un token. De plus, à chaque embedding de token on somme un embedding de position. Pour faire de la classification, un CLS token en position 0 est passé à une tête de classification. À l'instar des LLM, les représentations vectorielles des tokens vont être successivement passés à des couches de transformers. Un block de transformer est composé d'un mécanisme de "self-attention" (dans ce cas bidirectionnelle), d'un MLP, de connections résiduelles et des couches de normalisation.

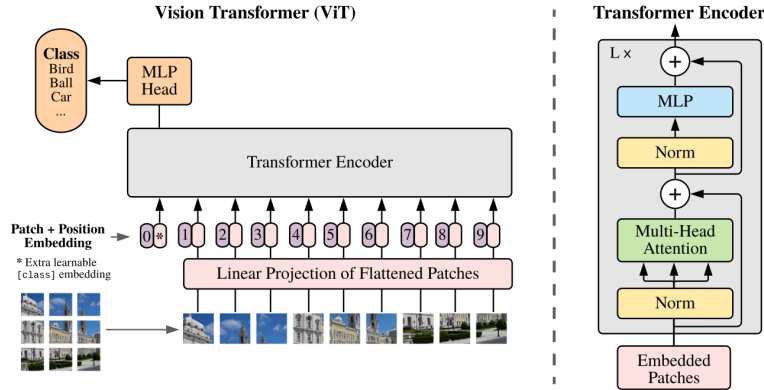


Figure 6: Schema d'un ViT (<https://arxiv.org/pdf/2010.11929>)

Les transformers, en faisant moins d'hypothèses sur la structure de l'entrée, permettent de capturer mieux des dépendences qui ne sont pas locales. En revanche, ils demandent généralement plus de données pour atteindre la même performance que les CNN. L'architecture du ViT est présentée dans la figure 6.

3 Comparaison de performances

Faire une comparaison juste entre deux architectures demande de prendre en compte plusieurs facteurs. D'une part, il faut que les deux modèles aient une "expressivité" similaire. A priori, un modèle avec 10 fois plus de paramètres permet d'approximer plus de fonctions et donc celui-ci il aurait un avantage significatif face à un modèle avec moins de paramètres. Pour cette raison nous allons comparer des CNN et des transformers ayant un nombre de paramètres similaires. D'autre part, évaluer un modèle sur un dataset donné ne permet pas immédiatement de mettre en évidence un "défaut" d'une architecture. En effet, il faudrait démontrer que toute configuration issue de cette architecture échoue dans ce dataset. Ceci permettrait de faire la différence entre un mauvais choix hyperparamètres (comme le learning rate) et un obstacle intrinsèque à l'architecture pour ce dataset. Néanmoins, cette approche n'est pas faisable dans la pratique. Ainsi, nous allons d'abord choisir un dataset, et une configuration d'un ViT et un CNN ayant une bonne performance sur le dataset et puis nous allons modifier le dataset de manière à que seulement l'une des deux performances se dégrade. Cette approche permet de justifier que la mauvaise performance de l'une des deux architectures n'est pas due à une mauvaise configuration mais à la difficulté supplémentaire rajoutée dans le dataset.

De plus, un modèle pré-entraîné peut induire des biais dans les résultats. Par

exemple, si le modèle a été entraîné avec un dataset similaire, voire le même. Pour cette raison nous allons entraîner de zero les modèles.

3.1 Sélection des datasets

3.1.1 Dataset A: CNN perdant

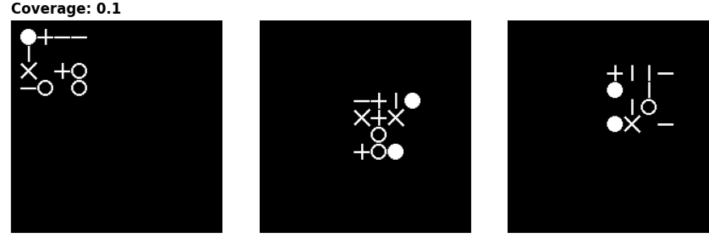
Étant donné que les CNN font une hypothèse de localité (i.e. les informations locales suffisent à déterminer le label) pour trouver un dataset où les CNN sont moins performants que les ViT, il est pertinent de considérer un cas où les informations sont dispersées dans toute image. Ainsi, nous considérons un dataset où l'objectif est de savoir si l'image contient ou non tout les symboles. Cette tâche demande de considérer l'ensemble de l'image car deux symboles peuvent être dans des côtés opposés de l'image et donc le kernel n'observera que un à la fois. Pour rendre la tâche plus difficile on choisit des symboles similaires entre eux (comme le signe '+', '-', 'X', etc.). Une image composé que des signes + aura le label "False" car il manque les symboles '-', 'X', etc.

Cette tâche devrait être plus difficile pour un CNN lorsque les symboles sont dispersés dans toute l'image. Dans ce cas, il faut que le CNN agrège l'information de toute l'image; et donc le label ne dépend pas que des informations locales. Les ViT, ayant la capacité à voir toute l'image simultanément, devrait avoir une performance supérieure. Dans notre dataset un paramètre permet de contrôler la dispersion des symboles dans notre image, ce qui permettra de comparer les performances en fonction des distances entre les symboles pour chaque modèle. Des exemples d'entraînement sont données dans la figure 8a.

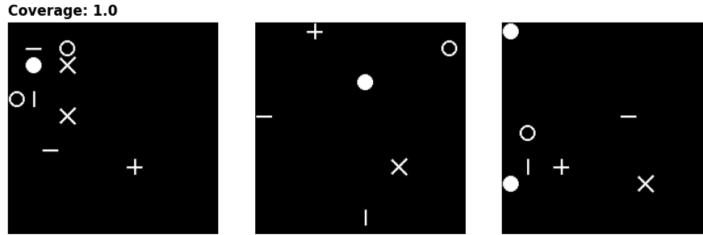
3.1.2 Dataset B: ViT perdant

Au contraire des CNN, les ViT "regardent" de manière fixe chaque zone de l'image. C'est-à-dire qu'il est plus facile pour un ViT que pour un CNN de corréler la position absolue d'un signal avec le label de l'image. Sous cette hypothèse, nous pouvons considérer un dataset où l'objectif est de distinguer un rectangle d'un cercle. Nous allons rajouter dans le train et validation set un point qui sert de signal et dont la position sera plus ou moins corrélé avec le label à travers un paramètre. Néanmoins, dans le test set la position du point sera indépendante du label.

Ainsi, on peut espérer que le ViT va rapidement corréler la présence du point dans un patch avec le label; alors que le CNN, étant donné ses propriétés d'invariance par translation, sera moins sensible à la position du point. En effet, du point de vu du CNN la forme (rectangle ou cercle) donne plus d'information sur le label que le point car le point est observé par le kernel de la même manière indépendamment de sa position. Par conséquent, on peut espérer que le CNN aura une performance élevée dans le train, validation et test set alors que le ViT va échouer dans le test set. Cette différence devrait être d'autant plus notable que la corrélation entre le label et la position du point est importante.



(a) Exemples d'entraînement avec un paramètre de "coverage" faible (symboles peu dispersés). L'image à gauche a pour label "False" car le symbole représentant une ligne verticale n'est pas présent. L'image à droite a pour label "True" car tous les symboles sont présents.



(b) Exemples d'entraînement avec un paramètre de "coverage" maximal (symboles très dispersés)

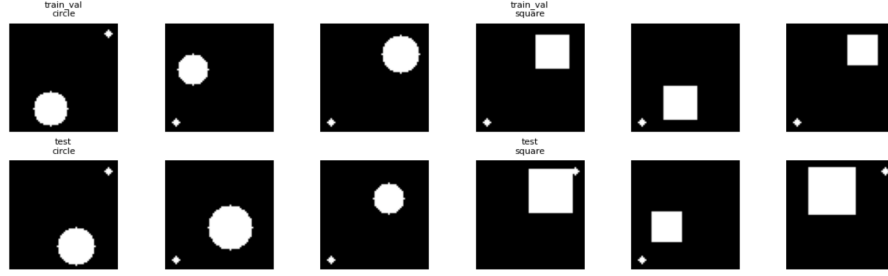
Figure 7: Exemples d'entraînement du dataset A

On pourrait considérer que ce cas est très peu réaliste, mais il est courant que les modèles apprennent à prendre en compte le signal incorrect pour déterminer un label. Par exemple, si on veut identifier des animaux dans des images, l'heure à laquelle l'image a été prise donne une information importante sur le label (certains animaux sont plus susceptibles d'apparaître la nuit), en revanche ce n'est pas un signal que l'on veut nécessairement prendre en compte.

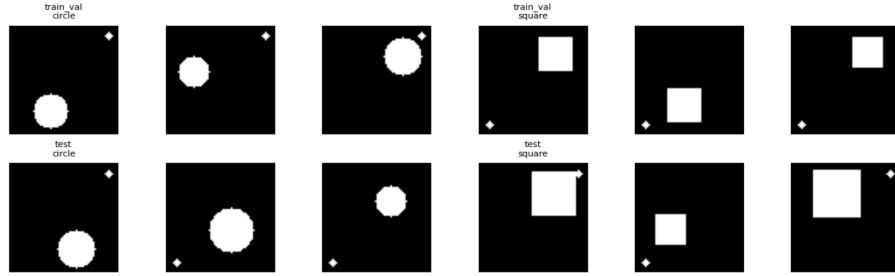
3.2 Entraînement des modèles

La première étape de la comparaison consiste à entraîner un ViT et un CNN avec des bonnes performances dans le cas de base. Les hyperparamètres (learning rate, dimension des MLP, etc.) ont été choisis de manière empirique, i.e. en partant des hyperparamètres de référence et en le modifiant jusqu'à avoir des résultats satisfaisants. Cependant, ceci a été fait sous la contrainte que les modèles aient un nombre de paramètres similaires.

Ensuite, une fois un ViT et un CNN de référence a été entraîné, on modifie le paramètre du dataset (la dispersion des symboles dans le premier dataset et la corrélation entre le label et la position d'un signal dans le cas du deuxième



(a) Exemples d'entraînement avec un paramètre de corrélation entre le label et la position du point faible. Sur la première ligne (train set) la position du point ne fournit pas d'informations sur la classe. Sur la deuxième ligne (test set) la position du point ne fournit pas d'informations sur la classe.



(b) Exemples d'entraînement avec un paramètre de corrélation entre le label et la position du point maximal. Sur la première ligne (train set) la position du point permet de déterminer la classe. Sur la deuxième ligne (test set) la position du point ne fournit pas d'informations sur la classe.

Figure 8: Exemples d'entraînement du dataset B

dataset) et on reentraîne ce même modèle sur cette variante du dataset. L'entraînement s'arrête lorsque le nombre maximal de epochs est atteint ou lorsque la validation loss cesse de s'améliorer après un certain nombre de epochs (early stopping). De plus, l'entraînement se fait sur GPU pour accélérer les expériences (notamment lorsqu'on entraîne les ViT).

Dans le dataset A, les modèles ont environ 180K paramètres alors que dans le dataset B ils ont environ 4.5K paramètres.

3.3 Résultats

Dans le cas du dataset A, les résultats sont l'inverse de ce que l'on attendait. En effet, dans la figure 9, on observe que la performance des CNN se dégrade lorsque les symboles sont plus proches entre eux. Une possible explication pour

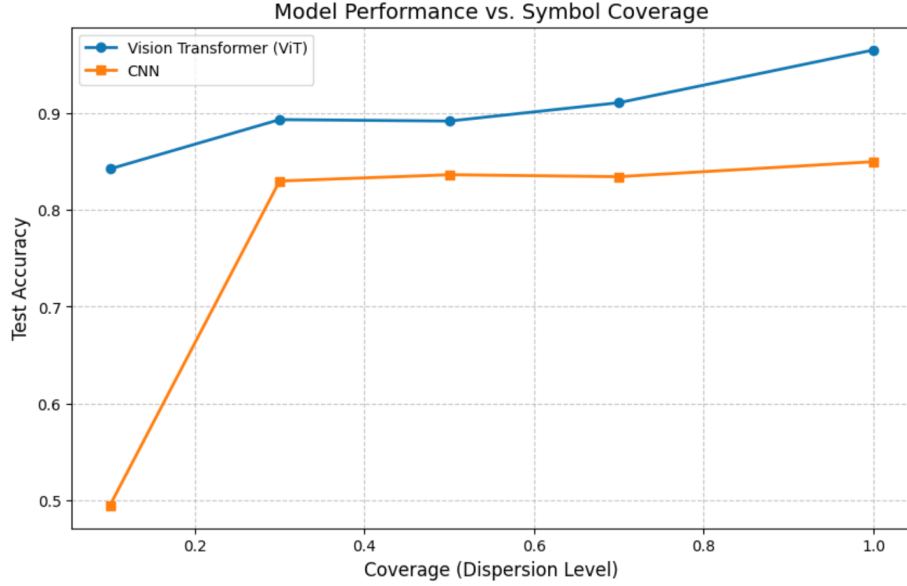


Figure 9: Test accuracy en fonction du paramètres "coverage" sur le dataset A

ce résultat est que plus isolé est un point, plus facile il est de le compter avec un kernel. En effet, si plusieurs symboles sont proches entre eux un kernel qui est translaté aura plus de difficultés à distinguer chaque symbole individuellement. Néanmoins, la performance des CNN est toujours inférieure à la performance ViT.

Dans le cas du dataset B, les résultats sont présentés dans la figure 10. On observe précisément ce qu'on aurait pu espérer: lorsque la corrélation entre la position du point et le label augmente, le ViT apprend cette relation et par conséquent échoue sur test set. Ceci met en évidence que le ViT apprend la position du point plutôt que la forme dans l'image (le vrai label). De plus, le CNN semble ne pas utiliser la position du point étant donné que sa performance ne change pas en fonction du paramètre du dataset. Le seul cas pathologique est lorsque la position du point détermine entièrement le label, et à ce moment là les deux performances diminuent significativement sur le test set.

3.4 Limitations

Ces expériences auraient pu être améliorées de plusieurs manières. D'abord, une comparaison plus "juste" entre les modèles aurait été de comparer les FLOPs nécessaires à l'entraînement plutôt que le nombre de paramètres. En effet, les ViT demandent plus de ressources lors de l'entraînement que les CNN (si le nombre de paramètres est le même). De plus, pour mettre en évidence les faiblesses de ViT contre les CNN on aurait pu considérer comme paramètre la

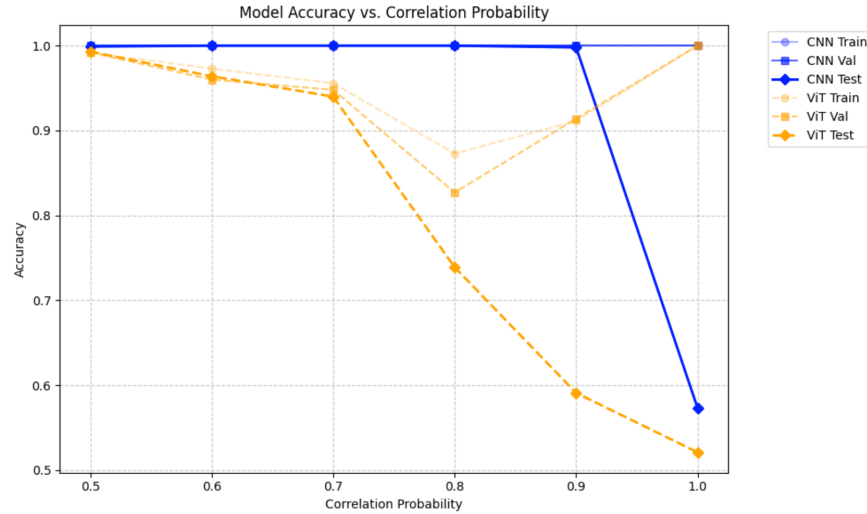


Figure 10: Test accuracy en fonction du paramètre de corrélation entre le label et la position du point dans le dataset B

taille du dataset. En effet, les ViT demandent de plus de données que les CNN en général et donc on peut s'attendre à que la performance de ViT soit supérieure aux CNN pour des datasets de grande taille. Néanmoins, ces expériences sont difficiles à mener sans des ressources computationnelles importantes.

4 Conclusion

En conclusion, nous avons mis en évidence comment comprendre l'architecture des ViT et des CNN permet d'identifier leurs limites ainsi que potentiellement anticiper si un méthode va échouer sur un dataset donnée. Ce travail pourrait être approfondi avec un étude plus fine des ressources utilisés par chaque modèle et en explorant des datasets plus variés.

5 Code

Le code est accesible sur le lien suivant : <https://github.com/sixtocerna/DL-Projet-M2>